Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#_09__)

Implementation of a Register File using ModelSim

Student Name: _Eesha Rehan_____ Roll No: 24i-6027_____ Section: B___

Date performed: __21st October _____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1+Task 2:

Description:

In this task we have to design the 4x4 register file in which we make module of decoder 2 by 4 in which we take two inputs and 4 outputs and those two inputs will decide where to enter in which register and this is done when write enable is active after that mux will have 4 inputs and one output that will show the register value that is stored and it is based on selection lines after that we made a module of register where we load value at positive edge of clock and when load is 1. After that we made register file module in which we call all the modules and tested it in testbench.

Code:

```
module decoder_2to4(output reg [3:0] dec_out, input [1:0] wr, input write_enable);

always @(*) begin

 if(write_enable) begin

   case(wr)

    2'b00: dec_out = 4'b0001;

    2'b01: dec_out = 4'b0010;

    2'b10: dec_out = 4'b0100;

    2'b11: dec_out = 4'b1000;

    default: dec_out = 4'b0000;

   endcase

 end else begin

   dec_out = 4'b0000;

 end

end

endmodule



module mux4_1(output reg [3:0] out, input [3:0] in0, in1, in2, in3, input [1:0] sel);
```

```
always @(*) begin
 case(sel)
   2'b00: out = in0;
   2'b01: out = in1;
   2'b10: out = in2;
   2'b11: out = in3;
   default: out = 4'b0000;
 endcase
end
endmodule


module register_4bit(output reg [3:0] q, input [3:0] d, input load, input clk);
initial begin
q=4'b0000;
end
always @(posedge clk) begin
 if(load)
   q <= d;
end
endmodule



module register_file_4x4(output [3:0] data_out1, data_out2,
```

```
                input [3:0] data_in,

                input [1:0] wr,

                input write_enable,

                input [1:0] rd1, rd2,

                input clk);



wire [3:0] dec_out;

wire [3:0] q0, q1, q2, q3;



decoder_2to4 decoder(dec_out, wr, write_enable);




register_4bit reg0(q0, data_in, dec_out[0], clk);

register_4bit reg1(q1, data_in, dec_out[1], clk);

register_4bit reg2(q2, data_in, dec_out[2], clk);

register_4bit reg3(q3, data_in, dec_out[3], clk);



mux4_1 mux1(data_out1, q0, q1, q2, q3, rd1);

mux4_1 mux2(data_out2, q0, q1, q2, q3, rd2);



endmodule

module testbench();

  reg [3:0] data_in;
```

```
reg [1:0] wr, rd1, rd2;

reg write_enable, clk;

wire [3:0] data_out1, data_out2;

register_file_4x4 uut (

 .data_out1(data_out1),

 .data_out2(data_out2),

 .data_in(data_in),

 .wr(wr),

 .write_enable(write_enable),

 .rd1(rd1),

 .rd2(rd2),

 .clk(clk)

);

initial

begin

clk=0;

forever

#5 clk = ~clk;

end

initial begin

clk=0;

write_enable=0;

wr=0;
```

```
rd1=0;

rd2=0;

data_in=0;

#10;

 write_enable = 1; wr = 2'b00; data_in = 4'b1010; #10;

 write_enable = 1; wr = 2'b01; data_in = 4'b0110; #10;

 write_enable = 1; wr = 2'b10; data_in = 4'b0011; #10;

 write_enable = 1; wr = 2'b11; data_in = 4'b1101; #10;



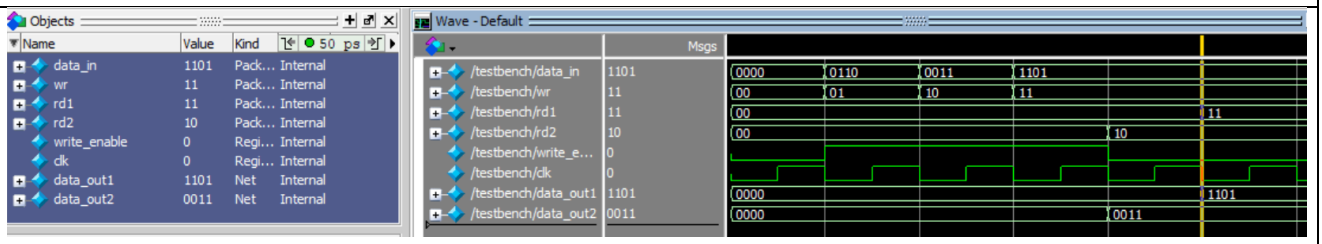
 write_enable = 0; rd1 = 2'b00; rd2 = 2'b10; #10;

rd1=2'b11; #10;

end

endmodule
```

Output:



Task 3:

Description:

In this task we designed 32 bit register file by using the module of decoder in which we used 5 bit input that gives 32 bit output and we designed its test case by doing left shift according to the write enable input ,using mux 32x1 where we initialize 32 inputs and 5 selections line that are giving one output then we make register file32bit module in which we call all of these module and then test it by using testbench.

Code:

```
module decoder_5to32(output reg [31:0] dec_out,

                input [4:0] wr,

                input write_enable);



always @(*) begin

 if(write_enable)

   dec_out = 32'b1 << wr;

 else

   dec_out = 32'b0;

end



endmodule



module mux32_1(output reg [31:0] out,

           input [31:0] in0, in1, in2, in3, in4, in5, in6, in7,

                   in8, in9, in10, in11, in12, in13, in14, in15,

                   in16, in17, in18, in19, in20, in21, in22, in23,

                   in24, in25, in26, in27, in28, in29, in30, in31,

           input [4:0] sel);



always @(*) begin

 case(sel)

   5'd0:  out = in0;
```

```
5'd1:  out = in1;

5'd2:  out = in2;

5'd3:  out = in3;

5'd4:  out = in4;

5'd5:  out = in5;

5'd6:  out = in6;

5'd7:  out = in7;

5'd8:  out = in8;

5'd9:  out = in9;

5'd10: out = in10;

5'd11: out = in11;

5'd12: out = in12;

5'd13: out = in13;

5'd14: out = in14;

5'd15: out = in15;

5'd16: out = in16;

5'd17: out = in17;

5'd18: out = in18;

5'd19: out = in19;

5'd20: out = in20;

5'd21: out = in21;

5'd22: out = in22;

5'd23: out = in23;
```

```
   5'd24: out = in24;

   5'd25: out = in25;

   5'd26: out = in26;

   5'd27: out = in27;

   5'd28: out = in28;

   5'd29: out = in29;

   5'd30: out = in30;

   5'd31: out = in31;

   default: out = 32'b0;

  endcase

end



endmodule



module register_32bit(output reg [31:0] q,

                      input [31:0] d,

                      input load,

                      input clk,

                      input rst);



always @(posedge clk or posedge rst) begin

  if(rst)

    q <= 32'b0;
```

```
  else if(load)
    q <= d;
end


endmodule


module registerfile32bit(
    output [31:0] rdata1, rdata2,
    input [31:0] wdata,
    input [4:0] rs1, rs2, rd,
    input reg_write,
    input clk, rst
);


wire [31:0] dec_out;
wire [31:0] q0,q1,q2,q3,q4,q5,q6,q7,
          q8,q9,q10,q11,q12,q13,q14,q15,
          q16,q17,q18,q19,q20,q21,q22,q23,
          q24,q25,q26,q27,q28,q29,q30,q31;

// Decoder
decoder_5to32 decoder(dec_out, rd, reg_write);
```

```
assign q0 = 32'b0;



register_32bit reg1 (q1,  wdata, dec_out[1],  clk, rst);
register_32bit reg2 (q2,  wdata, dec_out[2],  clk, rst);
register_32bit reg3 (q3,  wdata, dec_out[3],  clk, rst);
register_32bit reg4 (q4,  wdata, dec_out[4],  clk, rst);
register_32bit reg5 (q5,  wdata, dec_out[5],  clk, rst);
register_32bit reg6 (q6,  wdata, dec_out[6],  clk, rst);
register_32bit reg7 (q7,  wdata, dec_out[7],  clk, rst);
register_32bit reg8 (q8,  wdata, dec_out[8],  clk, rst);
register_32bit reg9 (q9,  wdata, dec_out[9],  clk, rst);
register_32bit reg10(q10, wdata, dec_out[10], clk, rst);
register_32bit reg11(q11, wdata, dec_out[11], clk, rst);
register_32bit reg12(q12, wdata, dec_out[12], clk, rst);
register_32bit reg13(q13, wdata, dec_out[13], clk, rst);
register_32bit reg14(q14, wdata, dec_out[14], clk, rst);
register_32bit reg15(q15, wdata, dec_out[15], clk, rst);
register_32bit reg16(q16, wdata, dec_out[16], clk, rst);
register_32bit reg17(q17, wdata, dec_out[17], clk, rst);
register_32bit reg18(q18, wdata, dec_out[18], clk, rst);
register_32bit reg19(q19, wdata, dec_out[19], clk, rst);
```

```
register_32bit reg20(q20, wdata, dec_out[20], clk, rst);

register_32bit reg21(q21, wdata, dec_out[21], clk, rst);

register_32bit reg22(q22, wdata, dec_out[22], clk, rst);

register_32bit reg23(q23, wdata, dec_out[23], clk, rst);

register_32bit reg24(q24, wdata, dec_out[24], clk, rst);

register_32bit reg25(q25, wdata, dec_out[25], clk, rst);

register_32bit reg26(q26, wdata, dec_out[26], clk, rst);

register_32bit reg27(q27, wdata, dec_out[27], clk, rst);

register_32bit reg28(q28, wdata, dec_out[28], clk, rst);

register_32bit reg29(q29, wdata, dec_out[29], clk, rst);

register_32bit reg30(q30, wdata, dec_out[30], clk, rst);

register_32bit reg31(q31, wdata, dec_out[31], clk, rst);




mux32_1 mux1(rdata1, q0,q1,q2,q3,q4,q5,q6,q7,

        q8,q9,q10,q11,q12,q13,q14,q15,

        q16,q17,q18,q19,q20,q21,q22,q23,

        q24,q25,q26,q27,q28,q29,q30,q31, rs1);



mux32_1 mux2(rdata2, q0,q1,q2,q3,q4,q5,q6,q7,

        q8,q9,q10,q11,q12,q13,q14,q15,

        q16,q17,q18,q19,q20,q21,q22,q23,

        q24,q25,q26,q27,q28,q29,q30,q31, rs2);
```

```
endmodule


module testbench();


reg clk, rst, reg_write;
reg [4:0] rs1, rs2, rd;
reg [31:0] wdata;
wire [31:0] rdata1, rdata2;



registerfile32bit uut(rdata1, rdata2, wdata, rs1, rs2, rd, reg_write, clk, rst);


// Clock generation
always #5 clk = ~clk;


initial
begin


   clk = 0;
   rst = 0;
   reg_write = 0;
   rs1 = 0;
```

```
rs2 = 0;

rd = 0;

wdata = 32'd0;




rst = 1; #10;

rst = 0; #5;




@(posedge clk);

reg_write = 1; rd = 5'd1; wdata = 32'd6;



@(posedge clk);

reg_write = 1; rd = 5'd2; wdata = 32'd3;



@(posedge clk);

reg_write = 1; rd = 5'd3; wdata = 32'd13;



// Read values

@(posedge clk);

reg_write = 0; rs1 = 5'd1; rs2 = 5'd2;   //  rdata1=6, rdata2=3



@(posedge clk);
```

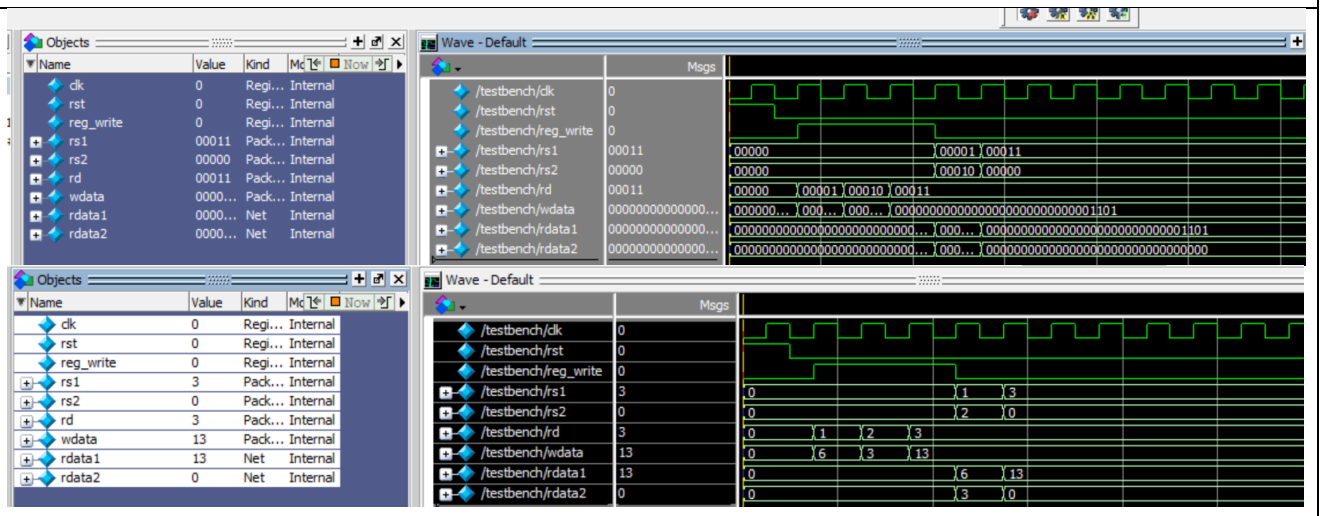
```
    rs1 = 5'd3; rs2 = 5'd0;              //  rdata1=13, rdata2=0


    #20;


end


endmodule
```

Output:

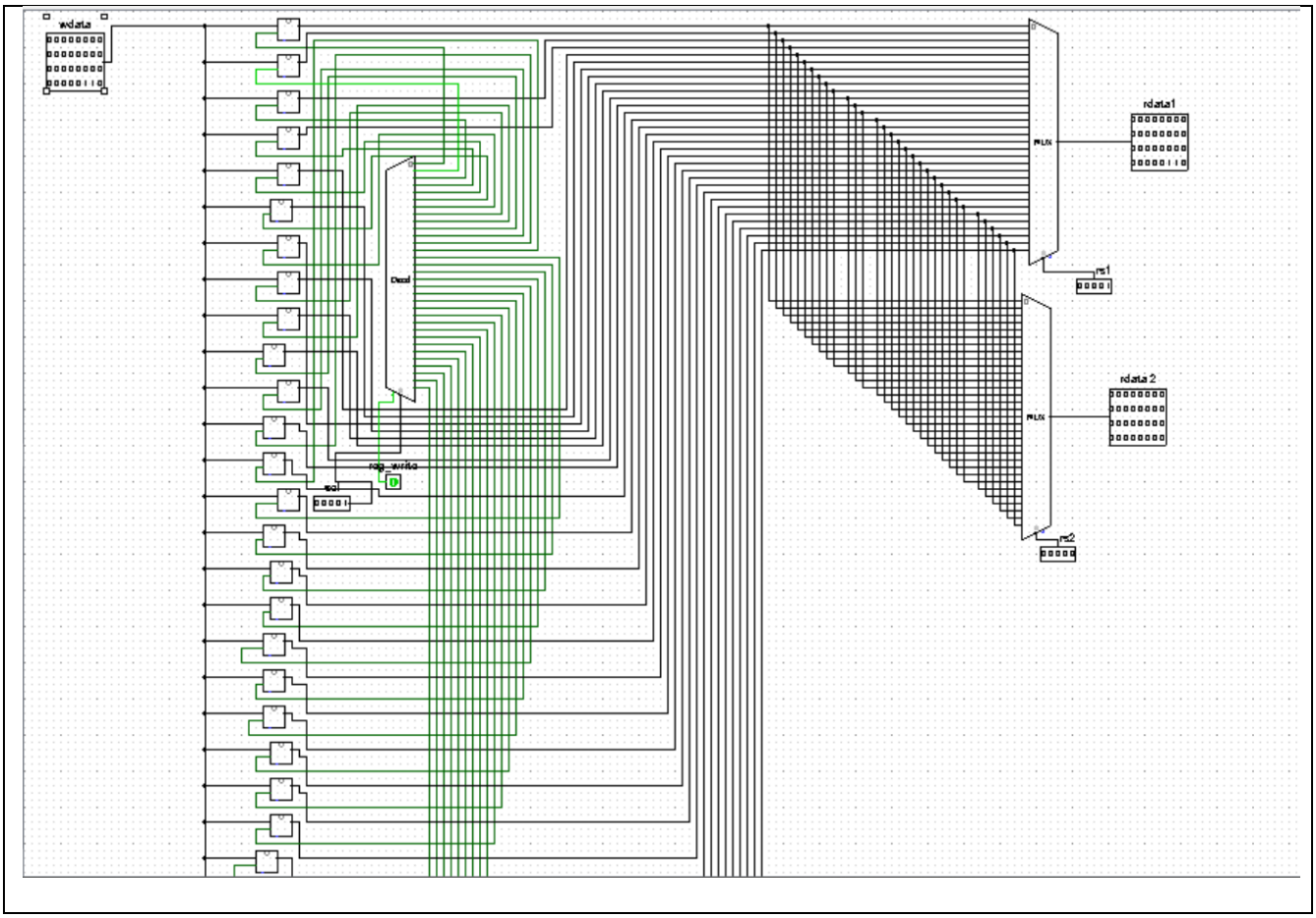


Task 4:

Description:

Following is the logisim diagram:

Output:


wdata
Decd
reg_write
wsel
rdata1
MUX
rs1
rdata 2
MUX
rs2

